

Checkpoint — Bandit 00 a 15 (repasso completo)

Nota de repaso en **español**, deliberada. El resto del vault está en inglés; esto es material de consolidación para leer sin ordenador delante.

Cubre los dieciséis niveles del hito de Mes 1. Las contraseñas están en las notas individuales de cada nivel; aquí lo que interesa es **qué enseña cada uno y qué se aprendió peleándolo**.

PARTE 1 — Cómo funciona Bandit

La estructura del juego

Bandit es un wargame de OverTheWire: 34 niveles donde cada uno esconde la contraseña del siguiente. Se juega enteramente por SSH.

```
ssh -p 2220 banditN@bandit.labs.overthewire.org
```

- Puerto **2220**, no el 22 estándar
- Usuario `banditN`, contraseña la obtenida en el nivel anterior
- El nivel 0 usa contraseña `bandit0`

Reglas del entorno que hay que conocer

El directorio `home` es de solo lectura. No puedes crear ficheros en `~`. Para trabajos intermedios:

```
mkdir -p /tmp/nombre_dificil_de_adivinar && cd /tmp/nombre_dificil_de_adivinar
```

`/tmp` es compartido y no listable. Tiene permiso de escritura y ejecución pero no de lectura para otros, así que puedes crear directorios pero no ver los de los demás. De ahí que el banner pida nombres difíciles de adivinar: no pueden *listar*, pero sí pueden *entrar* si aciertan el nombre.

Las contraseñas viven en `/etc/bandit_pass/banditN`, con permisos `-r-----` y propietario `banditN`. Solo ese usuario puede leerlas. Es el mecanismo que hace funcionar todo el juego.

El banner de bienvenida es el mismo para los 34 niveles. Menciona flags de compilador (`-m32`, `-fno-stack-protector`), ASLR desactivado y herramientas como `gef`, `pwndbg` y `radare2`. Nada de eso aplica hasta el nivel 25 largo, cuando empiezan los retos de explotación de binarios. Ignóralo hasta entonces.

No se puede conectar entre niveles desde dentro del servidor. OverTheWire bloquea SSH desde localhost para ahorrar recursos. Si necesitas conectarte a otro nivel, sal primero.

PARTE 2 — Recorrido por bloques

Bloque 1 · Niveles 00–02 — Conexión y nombres difíciles

Nivel 00 → 01

Conectar por SSH y leer el fichero `readme` del `home`. Es el nivel de calentamiento: comprobar que la conexión funciona y que sabes usar `ls` y `cat`.

Nivel 01 → 02 — El fichero llamado `-`

El fichero se llama literalmente un guion. `cat -` no da error: se queda esperando entrada del teclado, porque por convención Unix un guion suelto significa "lee de stdin".

```
cat ./-          # la ruta relativa lo desambigua
cat /home/bandit1/- # la absoluta también
```

Lo que no funciona: las comillas. `cat "-"` sigue pasándole un guion al programa. Las comillas protegen del **shell** (separación de palabras, expansión de comodines), no de cómo el programa interpreta sus propios argumentos.

Y `--` tampoco es garantía: en `cat`, incluso `cat -- -` sigue leyendo de stdin. Para un fichero llamado exactamente `-`, la ruta explícita es la única técnica fiable.

Nivel 02 → 03 — Nombre con espacios

El fichero se llama `--spaces in this filename--`. Dos problemas a la vez:

- Empieza por guiones → se parsea como opciones
- Tiene espacios → el shell lo parte en varios argumentos

```
cat ./--spaces\ in\ this\ filename--
cat "./--spaces in this filename--"
```

El hábito práctico: escribe las primeras letras y pulsa **Tab**. El autocompletado escapa correctamente todo, mucho más fiable que poner barras a mano.

Concepto de fondo — argument parsing. Los programas no reciben una lista limpia de "opciones" y "ficheros": reciben un array plano de cadenas (`argv`), y **cada programa decide por su cuenta** qué es qué, casi siempre con la misma convención heredada: lo que empieza por `-` es una opción.

Esto no es un capricho académico. Es una clase de vulnerabilidad con nombre propio: **argument injection** (CWE-88). Un script que hace `tar -czf backup.tar.gz *` en un directorio donde alguien puede escribir permite inyectar opciones — un fichero llamado `--checkpoint-action=exec=sh` se le pasa a `tar` como si fuera una bandera legítima. Aplica igual a `chown`, `rsync`, `zip`.

Nota: [Linux - Argument Parsing and Special Filenames](#)

Bloque 2 · Niveles 03–05 — Encontrar lo que no se ve

Nivel 03 → 04 — Fichero oculto

Un fichero cuyo nombre empieza por punto dentro de un directorio. `ls` no lo muestra.

```
ls -a      # muestra ocultos
ls -la     # ocultos + formato largo
```

En Linux un fichero "oculto" no tiene ningún atributo especial: **es simplemente una convención** de que los nombres que empiezan por `.` no se listan por defecto. Se usa para configuración (`.bashrc`, `.ssh`, `.gitignore`), no para seguridad.

Nivel 04 → 05 — El único fichero legible

Diez ficheros llamados `-file00` a `-file09`; solo uno contiene texto legible.

```
file ./-file*
```

Dos ideas clave aquí:

El comodín se expande antes. El shell convierte `./-file*` en los diez nombres *antes* de que `file` se ejecute. Un solo comando, diez comprobaciones, y el `./` neutraliza el problema del guion inicial en todos a la vez.

`file` **no mira la extensión.** Inspecciona los primeros bytes del contenido. Cuando responde `data`, no es un error: es su respuesta honesta a "esto no coincide con ningún formato conocido".

Y la lección que se repite todo el juego: un comando que se ejecuta sin error no es un comando que ha encontrado lo correcto.

`cat` imprime bytes binarios sin quejarse — hizo exactamente lo que le pediste. Que el resultado tenga sentido para un humano es otra pregunta.

Nivel 05 → 06 — `find` con criterios combinados

El fichero está en algún lugar de `inhere/` y cumple tres condiciones: legible, 1033 bytes, no ejecutable.

```
find . -type f -size 1033c ! -executable
```

Los tests se combinan con **AND Implícito** — no hace falta operador entre ellos. `!` niega el siguiente test.

Nota: [Linux - find Command](#)

Bloque 3 · Niveles 06–07 — Búsqueda global y filtrado

Nivel 06 → 07 — Buscar en todo el sistema

```
find / -user bandit7 -group bandit6 -size 33c 2>/dev/null
```

`2>/dev/null` es imprescindible aquí. Buscar desde `/` genera cientos de errores de "Permiso denegado" que ahogan el resultado real. Ese `2>` redirige solo `stderr`; la salida buena sigue llegando a pantalla.

`/dev/null` es un fichero especial que descarta todo lo que se le escribe. Un agujero negro sin coste de disco.

Nivel 07 → 08 — `grep` filtra contenido

La contraseña está en `data.txt`, junto a la palabra `millionth`.

```
grep 'millionth' data.txt
cat data.txt | grep 'millionth' # equivalente
```

El error real de este nivel fue empezar buscando el fichero con `find`, cuando estaba en el home, y luego canalizar `find` hacia `grep`:

```
find / -name data.txt | grep 'millionth' # no devuelve nada
```

`grep` filtra las líneas que recibe. Lo que `find` imprime son rutas, no contenido. Estaba buscando la palabra "millionth" dentro de nombres de fichero.

Regla que nació aquí y se repitió después: `pwd` y `ls -la` antes de cualquier `find`. Reconocimiento barato antes que herramienta cara.

Bloque 4 · Niveles 08–12 — Procesamiento de texto y formatos

Este es el bloque más denso y tiene checkpoint propio: [Checkpoint - Niveles 08 a 12](#). Resumen aquí.

Nivel 08 → 09 — La línea única

```
sort data.txt | uniq -u
```

`uniq` solo compara cada línea con la inmediatamente anterior. No tiene memoria del fichero. Por eso `sort` no es decoración: agrupa las iguales para hacerlas adyacentes.

El método que se aplicó antes de resolver y que conviene conservar:

```
wc -l data.txt # 1001 líneas
uniq -u data.txt | wc -l # 981 - resultado inflado, señal de alarma
sort data.txt | uniq -c | head # cada línea aparece 10 veces
```

$100 \text{ líneas} \times 10 + 1 \text{ única} = 1001$. Estructura del fichero reconstruida antes de sacar la respuesta.

Nivel 09 → 10 — Texto dentro de un binario

```
strings data.txt | grep '==='
```

`grep` se negó a mostrar la coincidencia con `binary file matches`. **No es un error: es protección.** Volcar binario al terminal puede emitir bytes de control que corrompen la sesión.

`grep -a` lo fuerza y *funciona*, pero devuelve líneas binarias enteras: la respuesta está ahí y es ilegible. `strings` extrae las secuencias imprimibles (4+ caracteres por defecto) y las emite una por línea, convirtiendo el problema binario en un problema de texto.

Encontrar algo y poder leerlo son dos problemas distintos.

Nivel 10 → 11 — base64

```
base64 -d data.txt
```

Sin `-d`, `base64` **codifica** — y codificar base64 válido produce base64 válido más largo, sin ningún error.

Nivel 11 → 12 — ROT13

```
cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

`tr` sustituye caracteres **posicionalmente** entre dos conjuntos. Sus dos argumentos son los conjuntos, **nunca los datos**: `tr` solo lee de stdin y no acepta fichero de ninguna forma.

`N-ZA-Mn-za-m` es ROT13 expresado como mapeo: A→N, B→O... M→Z, N→A... Z→M. Ambos conjuntos miden 52 caracteres. Dígitos y espacios no aparecen en ninguno, así que pasan intactos.

ROT13 es su propio inverso (13 = 26/2): el mismo comando codifica y decodifica.

Nivel 12 → 13 — Compresión anidada

El nivel más largo del tramo. Un hexdump de un fichero comprimido nueve veces.

```
xxd -r hexdump.txt > stage1      # revertir el hexdump
file stage1                      # ¿qué es?
mv stage1 stage1.gz && gunzip stage1.gz
file stage1                      # ¿y ahora?
# ...repetir 8-9 veces
```

El ciclo: `file` → identificar → renombrar con la extensión → descomprimir → `file` otra vez.

<code>file</code> dice	extensión	comando	¿borra la entrada?
gzip compressed data	.gz	gunzip	sí
bzip2 compressed data	.bz2	bunzip2	sí
POSIX tar archive	.tar	tar -xf	no
ASCII text	—	cat	fin

Por qué hace falta renombrar: `gunzip` se niega a trabajar sin sufijo `.gz`. Es una comprobación de seguridad suya para saber cómo nombrar la salida, no la forma en que averigua el formato. `bunzip2` es más permisivo: descomprime igual y avisa, inventando un `.out`.

El atajo que lo evita: `gunzip -c fichero > salida`. Al escribir a stdout no tiene que inventar nombre y la comprobación sobra.

`tar` **rompe el patrón dos veces**: no borra la entrada, y extrae con el nombre que lleva dentro. Después de cada `tar -xf`, `ls` obligatorio.

Y el hexdump: es texto que representa binario. Por eso `cat` lo leía y `file` decía ASCII — ambos ciertos e inútiles. `xxd -r` traduce de vuelta a bytes.

Bloque 5 · Niveles 13–15 — Autenticación y red

Nivel 13 → 14 — Clave privada SSH

No hay contraseña: hay un fichero `sshkey.private` propiedad de `bandit14` pero con grupo `bandit13`, colocado ahí a propósito.

```
ssh -i sshkey.private -p 2220 bandit14@bandit.labs.overthewire.org
```

Autenticación por clave: demuestras identidad **poseyendo** un fichero, no sabiendo un secreto. El servidor guarda la mitad pública (`~/.ssh/authorized_keys`), tú la privada. La clave nunca viaja: el servidor manda un reto, el cliente lo firma, el servidor verifica la firma.

Por qué importa: una clave robada es peor que una contraseña robada — **no hay contraseña que cambiar**. Quien la tenga es ese usuario hasta que alguien revoque la pública en cada servidor que confía en ella.

Los cuatro tropezos del nivel, todos instructivos:

1. Los `<...>` de la documentación son marcadores, no sintaxis. `ssh -i <fichero>` hace que bash lea `<` y `>` como redirecciones.

2. Localhost bloqueado. Desde dentro de Bandit, el nombre público resuelve a `127.0.0.1`. Hay que salir y conectar desde tu máquina.

3. Permisos demasiado abiertos. SSH se niega a usar una clave que otros pueden leer. En Linux: `chmod 600`. En Windows, ACLs:

```
icacls clave /inheritance:r  
icacls clave /grant:r "$($env:USERNAME):(R)"
```

4. Formato inválido. Dos causas encadenadas: finales de línea CRLF de Windows, y falta del salto de línea final.

El método que resolvió el nivel: comparar tamaños. 2602 bytes en el servidor, 2601 en local. **Un byte.** Contar es barato y no miente.

Y un error de orden que enseña más de lo que parece: se restringieron los permisos *antes* de terminar de editar el fichero, y quedó bloqueado el acceso al propio fichero. Es la versión pequeña de aplicar una política restrictiva antes de acabar la configuración — en un firewall o un servidor remoto, eso te deja fuera de la máquina.

Nivel 14 → 15 — netcat, texto plano

```
cat /etc/bandit_pass/bandit14 | nc localhost 30000
```

`nc` abre una conexión TCP y la conecta a stdin y stdout. Sin protocolo, sin formato, sin cifrado. **Es una pipe que atraviesa la red.**

Conceptos:

- localhost** / `127.0.0.1` — interfaz de **loopback**, una interfaz virtual que apunta a la propia máquina. El tráfico nunca toca un cable.
- Puertos** — número de 16 bits (0-65535) que, junto con la IP, identifica un extremo de conexión. Por debajo de 1024 requiere root.

Aquí localhost sí funciona: el bloqueo del nivel 13 era específico de SSH entre niveles. No tiene nada que ver con conexiones TCP a servicios locales.

Confusión legítima del nivel: el enunciado pide "la contraseña del nivel actual". En todos los niveles anteriores esa cadena era a la vez la que usabas para entrar y la del usuario. Aquí se separan por primera vez, porque entraste con **clave**.

Nivel 15 → 16 — TLS

```
openssl s_client -connect localhost:30001
```

Mismo intercambio que el 14, pero cifrado.

El diagnóstico del nivel: `nc` al puerto 30001 devolvía **silencio y cierre**. Eso es información:

Comportamiento	Significado
Conexión rechazada	No hay nadie escuchando
Conexión que abre y cierra sin responder	Sí hay alguien, y no te entiende

Se parecen a simple vista y apuntan a sitios opuestos.

`openssl` no es un comando, es un paraguas. El primer argumento dice **qué hacer**: `s_client`, `x509`, `enc`, `genrsa`, `dgst`. Por eso `openssl localhost 30001` respondió `Invalid command 'localhost'`.

Del volcado del handshake, lo que merece recordarse:

- `depth=0` + subject e issuer idénticos → **certificado autofirmado**. `SnakeOil` es el nombre tradicional de Debian para certificados de prueba.
- `Verify return code: 18` → **cifrado sí, autenticación no**. Un navegador se habría negado.
- `TLS_AES_256_GCM_SHA384` → cifrado simétrico AES-256 en modo GCM (confidencialidad + integridad en una pasada), SHA-384 para derivación de claves.
- `X25519MLKEM768` → intercambio de claves **híbrido post-cuántico**. X25519 clásico + ML-KEM (antes Kyber). Protege frente a *harvest now, decrypt later*.
- `Compression: NONE` → deliberado. La compresión TLS se eliminó porque habilitaba el ataque **CRIME**.

La diferencia entre `nc` y `openssl s_client` es la diferencia entre HTTP y HTTPS. Hacer los niveles 14 y 15 seguidos es la demostración más limpia posible de qué añade TLS, porque todo lo demás se mantiene constante.

Y el matiz que más vale: **cifrado y confiable son dos propiedades distintas**. TLS las da con mecanismos separados que pueden fallar por su cuenta.

PARTE 3 — Las lecciones transversales

1. Un comando sin error no es un comando correcto

Es el hilo que recorre todo el tramo 8-15:

Comando	Parecía	Hacía
<code>uniq -u sin sort</code>	filtrar únicas	comparar solo adyacentes
<code>base64 sin -d</code>	decodificar	codificar otra vez
<code>sort fichero</code> en una pipe	ordenar lo que llega	ignorar la pipe
<code>cmd f > f</code>	transformar	vaciar antes de leer
<code>grep -a</code> sobre binario	encontrar	encontrar e ilegible
<code>nc</code> al puerto 30000 en L15	contraseña rechazada	puerto equivocado

Ninguno falló. Ninguno avisó.

2. El nombre no dice nada, los bytes sí

La extensión es convención humana. Lo que identifica un fichero es su **magic number**:

Bytes	ASCII	Formato
1f 8b		gzip
42 5a 68	BZh	bzip2
50 4b 03 04	PK	zip, docx , xlsx , pptx
4d 5a	MZ	ejecutable Windows
7f 45 4c 46	.ELF	ejecutable Linux
25 50 44 46	%PDF	PDF

Demostrado en la práctica: renombrar un tar a `.gz` y ver que `gunzip` responde `not in gzip format`.

3. Mirar antes de actuar

<code>pwd</code>	<code>&& ls -la</code>	# ¿dónde estoy y qué hay?
<code>file</code>	<code>fichero</code>	# ¿qué es realmente?
<code>head</code>	<code>-3 fichero</code>	# ¿qué pinta tiene?
<code>wc</code>	<code>-l fichero</code>	# ¿de cuánto hablamos?
<code>comando</code>	<code> head -30</code>	# ¿qué produce esta herramienta nueva?

Ninguno resuelve nada. Todos hacen que la respuesta sea fiable.

4. Contar es barato y no miente

Tres veces resolvió un problema:

- `wc -l` en el 8 para mapear la estructura del fichero
- `ls -la` en el 12 para detectar el fichero a cero bytes
- `wc -c` vs `.Length` en el 13 para encontrar el byte que faltaba

5. Leer el error entero

- `Not a directory` → no era un problema de ruta, era un fichero
- `binary file matches` → no era fallo, era éxito con la salida retenida
- `missing operand` → `tr` quería conjuntos, no datos
- `Invalid command 'localhost'` → faltaba el subcomando
- `Can't guess original name` → aviso, no error: había descomprimido bien

6. La documentación local va antes que el buscador

`--help` y `man` responden en un segundo, son de tu versión exacta y funcionan sin red. El rodeo con `-m32` salió de leer un banner en vez de leer `base64 --help`.

PARTE 4 — Referencia de comandos

Navegación e inspección

Comando	Uso	Trampa
<code>pwd</code>	dónde estoy	el reflejo previo a cualquier ruta relativa
<code>ls -la</code>	listar con ocultos	<code>-a</code> es lo que revela los <code>.fichero</code>
<code>cd</code>	cambiar directorio	<code>~</code> solo vale solo o seguido de <code>/</code>
<code>file</code>	identificar formato	lee bytes, ignora el nombre
<code>head -N</code> / <code>tail -N</code>	primeras/últimas líneas	<code>-c</code> cuenta bytes; <code>tail -F</code> para logs que rotan
<code>wc -l</code> / <code>-c</code> / <code>-w</code>	contar	validar cada etapa de una pipeline

Búsqueda

Comando	Uso	Trampa
<code>find ruta -tests</code>	buscar por criterios	sin <code>-name</code> , un argumento suelto es ruta de inicio
<code>grep patrón fichero</code>	filtrar líneas	filtra contenido , no rutas
<code>strings fichero</code>	texto en binarios	por defecto solo secuencias de 4+ caracteres

Tests de `find` útiles: `-name` `-type f|d` `-size 1033c` `-user` `-group` `-perm -4000` `! -executable` `-mtime -1` `-maxdepth 1`

Procesamiento de texto

Comando	Uso	Trampa
<code>sort</code>	ordenar	sin <code>-n</code> compara como texto: <code>"10" < "9"</code>
<code>uniq -u</code> / <code>-d</code> / <code>-c</code>	únicas / repetidas / contar	exige sort delante
<code>tr SET1 SET2</code>	sustituir caracteres	solo stdin, jamás fichero
<code>tr -d '\r'</code>	borrar caracteres	quita CRLF de Windows

Codificación y binarios

Comando	Uso	Trampa
<code>base64 -d</code>	decodificar	sin <code>-d</code> codifica
<code>xxd / xxd -r</code>	hexdump / revertir	nunca al mismo fichero
<code>gunzip / bunzip2</code>	descomprimir	borran la entrada ; <code>-c</code> lo evita
<code>tar -xf</code>	extraer	no borra; extrae con nombre interno

Red y autenticación

Comando	Uso	Trampa
<code>ssh -p 2220 user@host</code>	conectar	Bandit usa 2220, no 22
<code>ssh -i clave</code>	autenticar con clave	ruta relativa: depende de dónde estés
<code>chmod 600 clave</code>	permisos de clave	SSH se niega si otros pueden leerla
<code>nc host puerto</code>	TCP en crudo	host y puerto separados por espacio
<code>openssl s_client -connect host:puerto</code>	cliente TLS	aquí van unidos por dos puntos

Streams y redirección

Símbolo	Significado
<code>\ </code>	stdout izquierda → stdin derecha
<code>></code>	stdout a fichero (crea o vacía)
<code>>></code>	stdout a fichero (añade)
<code><</code>	stdin desde fichero
<code>2></code>	solo stderr
<code>2>/dev/null</code>	descartar errores
<code>&&</code>	ejecuta si lo anterior tuvo éxito
<code>\ \ </code>	ejecuta si lo anterior falló
<code>\$(cmd)</code>	sustituye por la salida de cmd

PARTE 5 — Autoevaluación

Sin mirar las notas:

Bloque 1-3

1. ¿Por qué `cat "-"` no funciona y `cat ./-` sí? ¿Qué hace realmente `cat -`?
2. ¿Qué es la argument injection y por qué `tar -czf b.tar.gz *` puede ser peligroso?
3. ¿Qué significa que `file` devuelva `data`?
4. ¿Para qué sirve `2>/dev/null` en un `find /` y por qué es casi obligatorio?
5. ¿Por qué `find / -name data.txt | grep 'x'` no busca dentro de los ficheros?

Bloque 4

6. ¿Por qué `uniq -u` sin `sort` da resultados incorrectos?
7. ¿Qué diferencia hay entre `grep -a` y `strings` sobre un binario?
8. ¿Qué pasa si ejecutas `base64` sin `-d` sobre datos ya codificados?
9. ¿Por qué `tr` no acepta un nombre de fichero?
10. ¿Qué queda en `f.txt` tras `xxd -r f.txt > f.txt` y en qué momento se pierde?
11. Un fichero se llama `backup.gz` pero `file` dice `POSIX tar archive`. ¿Quién tiene razón?
12. ¿Por qué `gunzip` exige la extensión `.gz` si no la usa para identificar el formato?

Bloque 5

13. ¿Qué es `authorized_keys` y por qué no contiene ninguna contraseña?
14. ¿Por qué SSH rechaza una clave privada con permisos demasiado abiertos?
15. ¿Qué diferencia hay entre una conexión rechazada y una que abre y se cierra sin responder?

16. ¿Qué significa `Verify return code: 18` y qué propiedad de TLS ha fallado?

17. ¿Por qué `nc host:puerto` falla pero `openssl s_client -connect host:puerto` funciona?

PARTE 6 — Dónde estás y qué viene

Hito de Mes 1 cerrado: Bandit 0→15. 

Objetivo de Fase 0: nivel 25 mínimo. Quedan diez niveles, previstos para el Mes 2.

Lo que viene en el 16-25 (para orientación, sin spoilers): escaneo de puertos y servicios, más SSL/TLS, gestión de claves, cron y ejecución programada, scripting, y las primeras cosas que huelen a explotación. `nmap` aparece pronto.

Lo que no hay que arrastrar: el reflejo de usar `find` para localizar ficheros que están en el directorio actual. Ocurrió en el 7 y en el 8. La corrección es `pwd` y `ls -la` primero.

Notas relacionadas

Niveles individuales: `Bandit - Level 00` ... `Bandit - Level 15`

Checkpoint del bloque denso: [Checkpoint - Niveles 08 a 12](#)

Conceptos:

- [Linux - Argument Parsing and Special Filenames](#)
- [Linux - File Type Detection](#)
- [Linux - find Command](#)
- [Linux - Piping and Redirection](#)
- [Linux - Sorting and Deduplication](#)
- [Linux - Extracting Strings from Binaries](#)
- [Linux - Sampling and Inspecting Output](#)
- [Linux - Encoding vs Encryption](#)
- [Linux - Nested Archives and Compression Layers](#)
- [Linux - Permissions & Process Management](#)

Scripts: [Script - Bandit 12 Decompression Loop](#) · [Script - Code Walkthrough \(Bash & Python\)](#)

Teoría relacionada: [Domain 2 - Network Security](#)